

Queremos simular un **taller** con un único **mecánico** y varios **coches** (clientes) que llegan aleatoriamente.

El mecánico solo puede reparar **un coche a la vez** y dispone de un **aparcamiento de espera** con capacidad limitada (por ejemplo, **5 plazas**).

Los coches que llegan cuando no hay plazas libres **se marchan**.

El programa debe reproducir correctamente la **lógica de sincronización** entre el mecánico y los coches, sin usar estructuras de datos avanzadas ni clases de concurrencia modernas.

### Reglas del sistema

1. Si no hay coches que reparar, **el mecánico se duerme** y espera a que llegue alguno.
2. Cuando llega un coche y el mecánico está dormido, **lo despierta**.
3. Si el mecánico está ocupado y hay plazas libres en el aparcamiento, **el coche aparca y espera**.
4. Si el aparcamiento está **lleno**, el coche **se va sin ser atendido**.
5. El mecánico **repara un coche a la vez**.
6. Cuando termina con un coche, si quedan coches esperando, **toma el siguiente** (orden FIFO simple). Si no, **vuelve a dormirse**.
7. El sistema debe funcionar **indefinidamente (o durante un tiempo razonable)**, sin interbloqueos ni errores de concurrencia.

### Requisitos técnicos

- Implementar el programa en **Java**.
- Utilizar únicamente las primitivas de sincronización básicas: `synchronized`, `wait()`, `notify()`, `notifyAll()`.
- No usar `java.util.concurrent`, `BlockingQueue`, ni colecciones concurrentes. Para la cola, si se quiere, puede usarse un arreglo circular o una lista simple protegida por el monitor.

### Actores del sistema

- **1 hilo Mecánico**

- **N hilos Coche** (por ejemplo, **10**), que llegan con tiempos aleatorios y con una “duración de reparación” también aleatoria (simulada con sleep).